Enterprise AI Engineering Bootcamp

From PoC to Production-Grade AI Systems

Revised version incorporating client observations

Prepared for advanced engineering teams working on AI PoCs, RAG, GenAI, CV, MLOps, agentic AI, MCP-enabled integrations, observability and production-grade AI architecture.

Recommended Format: 5 Full Days | Offline | Pune | Fully Hands-On

Revision Summary Based on Client Feedback

#	Client Observation	Curriculum Update Incorporated
1	Add hypothesis formulation and testing to Module 2.	Day 1, Module 2 now includes hypothesis formulation, null/alternate hypothesis design, test selection, Type I/II errors, effect size, power/sample size, statistical significance, and AI improvement experiments.
2	Day 2 should focus on types of AI architectures and selecting the right architecture by use case.	Day 2 has been redesigned around AI architecture patterns and a practical architecture-selection framework before moving into enterprise reference architecture, storage, data engineering and security.
3	Day 3 should include MCP and observability usage.	Day 3 now includes MCP concepts, MCP server/client patterns, tool/data integration, secure MCP contracts, RAG/LLM observability, traces, token/cost/latency monitoring, retrieval traces and failure analytics.
4	Cover the Skill concept.	A new Skill concept module has been added under Day 5, covering reusable AI/agent skills, skill packaging, instructions, examples, code/tools, governance, testing, versioning and enterprise reuse.

Program Positioning

This program is designed for advanced engineering teams who already understand AI fundamentals and have built multiple PoCs, but are now facing challenges in scaling those PoCs into reliable, secure, observable and production-grade enterprise AI systems.

The focus is not on learning AI from scratch. The focus is on helping experienced engineers improve the quality, reliability, scalability, explainability, security, observability and production readiness of AI systems.

The updated design explicitly addresses hypothesis-led AI improvement, architecture selection, MCP-enabled integrations, observability, and the Skill concept for reusable agent capabilities.

Recommended Format

Item	Details
Duration	5 full days
Location	Offline training in Pune
Daily Duration	7 to 8 hours
Audience	IIT graduate engineers from Mechanical Engineering and Computer Science backgrounds, already working on AI PoCs
Training Mode	Fully hands-on, architecture-led, use-case-driven, assignment-based and capstone-oriented
Primary Goal	Enable participants to redesign existing 60% accuracy PoCs into production-grade AI systems with a clear engineering path toward 95%+ accuracy or reliability, depending on the use case.

Important Positioning Note

For enterprise AI systems, “95% accuracy” must be defined carefully. A single accuracy number is not enough. The program will teach participants how to define, validate, test, monitor and continuously improve the right success metrics for each AI system.

AI System Type	Recommended Metrics
Classification	Accuracy, precision, recall, F1 score, ROC-AUC, calibration, confidence threshold
Defect Detection / CV	Mean average precision, false rejection rate, false acceptance rate, small object recall, edge-case coverage
RAG Systems	Context precision, context recall, faithfulness, answer relevance, citation accuracy, no-answer accuracy
Predictive Maintenance	Precision, recall, lead time, false alarm rate, missed failure rate, maintenance impact
GenAI Systems	Groundedness, hallucination rate, policy compliance, latency, cost per query, observability signals
Agentic AI Systems	Task completion rate, tool-call accuracy, skill invocation accuracy, escalation rate, safety violation rate

Business-Relevant Use Cases

The hands-on labs should be built around industrial use cases instead of generic examples. The following use cases are recommended for the training labs, daily assignments and final capstone.

Use Case 1: Industrial Service Knowledge Copilot

A role-based GenAI assistant that answers questions from technical manuals, service history, troubleshooting guides, maintenance logs, spare part documents and safety instructions.

- RAG architecture
- Layout-aware document processing
- Chunking strategy
- Tokenization
- Hybrid search and reranking
- Metadata filtering
- RBAC and secure retrieval
- Citation-backed answers
- Hallucination control
- MCP-enabled tool/data access
- RAG observability and evaluation

Use Case 2: Predictive Maintenance and Anomaly Triage

An AI system that analyzes equipment telemetry, compressor/vacuum system health data, alarms, service history and operational parameters to detect early failure signals.

- Time-series data engineering
- Feature engineering
- Anomaly detection
- Statistical thresholds
- Hypothesis testing
- Model monitoring
- Feedback loop
- Active learning
- Human-in-the-loop review
- MLOps pipeline

Use Case 3: Computer Vision for Industrial Quality Inspection

A CV model that detects defects, wear, leakage, incorrect assembly, corrosion, part misalignment, gauge readings or surface anomalies.

- Image classification
- Object detection
- Segmentation
- OCR for industrial labels and gauges
- Dataset quality
- Labeling strategy
- Defect taxonomy
- Active learning
- Model performance improvement
- Edge deployment considerations

Use Case 4: Agentic AI for Field Service Decision Support

A multi-agent system where different agents collaborate to diagnose equipment issues, retrieve manuals, check service history, recommend next actions and generate service reports.

- Agent design patterns

- Tool calling
- Planner-executor architecture
- Agent-to-agent communication
- MCP tool integration
- Skill design
- Human approval gates
- Guardrails
- Secure orchestration
- Audit logging and observability

Use Case 5: AI-Powered Engineering Research Assistant

A research-oriented assistant that scans technical papers, internal engineering notes, historical PoC results, product manuals and simulation data to recommend approaches for improving AI system accuracy.

- Research-backed AI development
- Literature-to-prototype workflow
- RAG over research papers
- Hypothesis generation
- Experiment tracking
- Reproducibility
- Technical decision documentation

Overall Learning Outcomes

1. Diagnose why an AI PoC is stuck at 60% accuracy and create a structured failure taxonomy.
2. Build reliable evaluation frameworks before improving the model or RAG pipeline.
3. Formulate testable AI improvement hypotheses and validate them using appropriate statistical methods.
4. Select the right AI architecture pattern based on use case, data type, latency, accuracy, explainability, risk, cost and deployment constraints.
5. Design scalable enterprise AI architecture with role-based access, storage layers, orchestration, security, monitoring and auditability.
6. Build production-grade RAG systems using layout-aware ingestion, tokenization strategy, metadata filtering, hybrid retrieval, reranking, MCP-enabled tool access and evaluation.
7. Instrument RAG and LLM systems with observability for retrieval quality, traces, token usage, latency, cost, failure analysis and production monitoring.
8. Improve computer vision systems using better data strategy, labeling, augmentation, error analysis, active learning and deployment-aware optimization.
9. Implement MLOps and LLMOps pipelines for continuous evaluation, monitoring, feedback, retraining and governance.
10. Design agentic AI systems with tool use, multi-agent collaboration, reusable Skills, guardrails, secure execution and human-in-the-loop control.
11. Translate research papers and emerging AI patterns into practical enterprise use cases and measurable experiments.
12. Create a measurable 30/60/90-day roadmap for improving each existing PoC toward production-grade quality.

Day -7: Pre-Training Preparation

This preparation should be completed before the Pune workshop so that the training can be mapped to the client's actual AI PoCs and pain points.

Inputs Required from Client

- List of existing AI PoCs
- Problem statement for each PoC

- Current architecture diagram, if available
- Current model type and technology stack
- Current accuracy or quality metrics
- Sample anonymized data
- Failure examples where the PoC gives wrong output
- Known business constraints
- Security and data access constraints
- Target production users and roles
- Current MCP/tool integration plans, if any
- Existing observability/logging approach, if any

Pre-Assessment Assignment

Participants submit a one-page PoC diagnosis sheet before the program.

- What problem does the PoC solve?
- Who is the user?
- What is the current accuracy or quality level?
- How is accuracy measured?
- What are the top five failure patterns?
- What data is used?
- What is the current architecture?
- What prevents it from scaling?
- Which architecture pattern seems most appropriate and why?
- What would production-ready mean?
- What tool or external system access is required?
- What observability data is currently captured?

Day 1: AI Accuracy Engineering, Statistics and PoC Diagnosis

Client feedback incorporated: Module 2 now explicitly covers hypothesis formulation and testing for AI system improvement.

Theme: Why AI PoCs stay at 60% accuracy and how to systematically move toward production-grade quality using evaluation, statistics, hypotheses and error analysis.

Module 1: From Model Accuracy to System Reliability

- Why AI PoCs fail in production
- Difference between demo accuracy and production accuracy
- Accuracy vs reliability vs trustworthiness
- Defining quality metrics by use case
- Business cost of false positives and false negatives
- Human-in-the-loop design
- Confidence thresholds
- Escalation logic
- Error budgets for AI systems

Hands-On Lab

Participants take one sample PoC and build a failure map covering data errors, labeling errors, retrieval errors, model errors, prompt errors, architecture errors, security errors, workflow errors and evaluation errors.

Module 2: Statistical Foundations, Hypothesis Formulation and Testing for AI Improvement

- Sampling strategy and representative evaluation datasets
- Train-validation-test split and stratified sampling
- Class imbalance and rare failure modes
- Precision, recall, F1 score, ROC-AUC and confusion matrix
- Confidence intervals and statistical significance
- Hypothesis formulation: business hypothesis, technical hypothesis and measurable AI hypothesis
- Null hypothesis and alternative hypothesis for AI improvement experiments
- Test selection: t-test, proportion test, chi-square test, non-parametric tests and paired evaluation where relevant
- Type I and Type II errors in AI model decisions
- Effect size, power and sample size considerations
- A/B testing and champion-challenger comparison for AI systems
- Inter-annotator agreement and label quality validation
- Bias and variance
- Drift detection
- Calibration and threshold tuning

Hands-On Lab

Teams formulate testable hypotheses for improving a low-performing PoC. Examples: hybrid retrieval improves context recall by 15%; better chunking reduces hallucination rate; augmentation improves CV recall for low-light images; active learning reduces false negatives. Teams select the right statistical test, define pass/fail criteria and design the evaluation dataset.

Module 3: Error Taxonomy and Root-Cause Analysis

- Why more data is not always the answer
- Data quality vs model quality
- Ambiguous labels and weak ground truth
- Poor feature representation
- Domain shift
- Edge cases and long-tail failures
- Measurement leakage and evaluation leakage
- Failure-based test creation
- Hypothesis backlog for systematic improvement

Hands-On Lab

Participants classify 100 failure examples into an error taxonomy and identify the top three improvement levers.

Day 1 Assignment

13. Current PoC baseline score
14. Failure taxonomy
15. Root-cause analysis
16. Metric definition for the 95% target
17. Three improvement hypotheses with expected impact
18. Recommended experiment design and statistical test for each hypothesis

Day 2: AI Architecture Patterns, Selection Framework, Data Engineering and Scalability

Client feedback incorporated: Day 2 now focuses on different types of AI architectures and how to select the right architecture based on use case before going deeper into enterprise reference architecture, storage, data engineering and security.

Theme: Moving from notebook-based PoCs to scalable, secure, maintainable enterprise AI systems by selecting the correct architecture pattern for the business problem.

Module 1: AI Architecture Patterns and Architecture Selection Framework

- Classical predictive ML architecture
- Time-series anomaly detection architecture
- Computer vision inspection architecture
- RAG/knowledge assistant architecture
- Fine-tuning architecture vs RAG vs prompt-only approach
- Agentic AI architecture
- Multimodal AI architecture
- Edge AI architecture for low-latency industrial environments
- Human-in-the-loop decision architecture
- Digital twin/simulation-assisted AI architecture
- Batch inference vs real-time inference
- Single-agent vs multi-agent architecture
- Cloud, on-premises and hybrid deployment options

Architecture Selection Criteria

- Business objective and user workflow
- Data type: documents, images, time-series, logs, structured records, audio or multimodal data
- Ground truth availability and label maturity
- Latency requirement
- Accuracy/reliability target
- Explainability and audit needs
- Security and role-based access requirements
- Level of autonomy and need for human approval
- Update frequency and drift risk
- Integration complexity
- Cost and infrastructure constraints
- Deployment location: cloud, plant, edge or hybrid

Hands-On Lab

Teams receive five Client Industry-style use cases and select the most appropriate architectural pattern for each. They justify why another architecture was rejected and what trade-offs were considered.

Module 2: Enterprise AI Reference Architecture

- User layer
- API layer
- Authentication and authorization
- Role-based access control
- Orchestration layer
- MCP/tool integration layer
- Model serving layer
- Data access layer
- Vector database
- Relational database
- Object storage

- Graph database
- Feature store
- Prompt registry
- Skill registry
- Model registry
- Monitoring and logging
- Audit trail
- Feedback loop
- Human approval layer

Hands-On Lab

Participants redesign a PoC architecture into an enterprise-ready architecture with user, data, model, orchestration, MCP/tool, observability and security layers.

Module 3: Storage Layers and Data Engineering for AI Systems

- Raw data layer
- Cleaned data layer
- Feature layer
- Embedding layer
- Vector index
- Metadata store
- Ground truth store
- Feedback store
- Audit store
- Model artifact store
- Prompt and policy versioning
- Batch ingestion
- Streaming ingestion
- Event-driven architecture
- Data contracts
- Schema validation
- Data lineage
- Data versioning
- Feature engineering
- Data quality checks
- Metadata tagging
- Access-controlled retrieval
- Data retention and masking

Hands-On Lab

Build a mini ingestion architecture for technical documents, telemetry data and feedback data. Participants identify where each data object should live and how it should be versioned, secured and monitored.

Module 4: Secure AI Architecture

- Identity and access management
- RBAC for GenAI systems
- Attribute-based access control
- Secure retrieval
- Data masking

- Secrets management
- API gateway
- Logging and audit
- Encryption at rest and in transit
- Secure tool calling
- Secure MCP server access
- Secure prompt and Skill handling
- Data leakage prevention

Hands-On Lab

Participants implement role-based retrieval logic for three roles: service engineer, product engineer and customer support user.

Day 2 Assignment

19. Architecture pattern selected for each team PoC
20. Architecture selection rationale and trade-off analysis
21. Target architecture diagram
22. Storage layer design
23. Data flow diagram
24. Security and access control design
25. Scalability risks and mitigation plan

Day 3: Production-Grade GenAI, RAG, Tokenization, MCP and Observability

Client feedback incorporated: Day 3 now includes MCP usage and observability for RAG/LLM systems, including tool/data integration, traces, cost, latency, retrieval quality and failure analytics.

Theme: Designing RAG systems that are accurate, layout-aware, secure, measurable, observable and integration-ready.

Module 1: Tokenization and Context Engineering

- What tokens are
- Tokenization and cost
- Context window limitations
- Prompt compression
- Chunk size impact
- Overlap strategy
- Lost-in-the-middle problem
- Context stuffing vs retrieval
- Token budgeting
- Multi-document context handling
- Token impact of tools, schemas, MCP calls, Skills and images

Hands-On Lab

Participants analyze how different chunk sizes, prompts and tool schemas change cost, latency and answer quality.

Module 2: Document Ingestion for Industrial RAG

- PDF parsing
- Layout-aware extraction
- Tables
- Diagrams
- Manuals

- Maintenance instructions
- OCR
- Image-text extraction
- Metadata tagging
- Section hierarchy
- Version control of documents
- Product-family-aware indexing

Hands-On Lab

Participants ingest a technical manual and compare naive text extraction, layout-aware extraction, table-preserving extraction and metadata-rich extraction.

Module 3: Retrieval Architecture

- Dense retrieval
- Sparse retrieval
- Hybrid search
- Reranking
- Metadata filtering
- Query rewriting
- Query expansion
- Multi-hop retrieval
- Parent-child chunking
- Hierarchical retrieval
- Graph RAG
- Self-checking RAG
- Retrieval fallback
- No-answer detection

Hands-On Lab

Build a RAG system over technical manuals and service notes, then compare multiple retrieval strategies.

Module 4: MCP for Tool and Data Integration

- Why MCP matters for enterprise AI applications
- MCP client and server roles
- Tools, resources and prompts exposed through MCP
- When to use MCP vs direct API integration
- MCP server contracts and tool descriptions
- Connecting RAG systems to external systems through MCP-style interfaces
- Identity, authorization and tool access boundaries
- Tool-call logging and auditability
- MCP security risks and mitigation considerations

Hands-On Lab

Teams design an MCP-enabled integration plan for a knowledge assistant that can retrieve technical documents, query a service-ticket store and call a diagnostic lookup tool with clear contracts and security boundaries.

Module 5: RAG Evaluation and Observability

- Context precision
- Context recall
- Faithfulness

- Answer relevance
- Citation accuracy
- Hallucination rate
- No-answer accuracy
- Groundedness
- Human evaluation
- Synthetic test-set generation
- Regression testing for RAG
- LLM traces and spans
- Prompt versioning and model versioning
- Retrieval traces
- Tool-call traces
- Token, latency and cost tracking
- User feedback capture
- Error clustering and root-cause dashboards
- Production alerts for quality regression

Hands-On Lab

Participants evaluate the same RAG system using multiple retrieval strategies, then design an observability dashboard showing retrieval quality, groundedness, latency, token usage, cost, tool-call failures and hallucination indicators.

Module 6: RAG Accuracy Improvement Patterns

- Better chunking
- Better metadata
- Better embeddings
- Hybrid retrieval
- Reranking
- Query decomposition
- Prompt grounding
- Answer verification
- Citation enforcement
- Domain glossary
- Knowledge graph
- Human feedback
- Failure-based test creation

Hands-On Lab

Teams improve a low-performing RAG pipeline and measure before-after performance using evaluation metrics and observability signals.

Day 3 Assignment

26. RAG architecture
27. Tokenization strategy
28. Chunking and metadata strategy
29. Retrieval benchmark
30. MCP integration plan and tool contract
31. RAG evaluation dashboard
32. Observability dashboard design
33. Top five changes that improved answer accuracy

Day 4: Computer Vision, Data Engineering, MLOps, Active Learning and Feedback Loops

Theme: Improving industrial CV and ML systems through data quality, experimentation, monitoring, active learning, feedback loops and continuous learning.

Module 1: Computer Vision for Industrial Use Cases

- Image classification
- Object detection
- Segmentation
- OCR
- Gauge reading
- Defect detection
- Surface anomaly detection
- Visual inspection
- Model selection
- Dataset size vs dataset quality
- Labeling guidelines
- Augmentation strategy
- Edge cases
- Lighting variation
- Camera angle variation
- Production environment variation

Hands-On Lab

Participants work on an industrial defect-detection dataset and identify why the model fails.

Module 2: CV Error Analysis and Hypothesis-Driven Improvement

- False positives and false negatives
- Confusion between defect classes
- Small object detection
- Occlusion
- Low contrast
- Poor labeling
- Class imbalance
- Domain shift
- Production drift
- Threshold tuning
- Human review process
- Hypothesis formulation for CV improvement
- Statistical comparison before and after improvement

Hands-On Lab

Participants create a CV error-analysis report, formulate improvement hypotheses and propose data/model/threshold actions.

Module 3: MLOps for Production AI Systems

- Experiment tracking
- Dataset versioning
- Model versioning

- Feature versioning
- CI/CD for ML
- Continuous training
- Model registry
- Model serving
- Rollback strategy
- Canary release
- Shadow deployment
- Batch inference
- Real-time inference
- Monitoring
- Drift detection
- Retraining triggers
- Reproducibility

Hands-On Lab

Participants create an MLOps pipeline for a predictive maintenance or CV model.

Module 4: Active Learning and Feedback Loop

- Human-in-the-loop learning
- Uncertainty sampling
- Error-based sampling
- Edge-case mining
- Feedback capture
- Label correction
- Expert review
- Continuous evaluation
- Feedback-to-retraining workflow
- Production learning loop

Hands-On Lab

Participants build a feedback loop where low-confidence predictions are routed for human review and added back into the training set.

Day 4 Assignment

34. CV or ML error analysis
35. Improvement hypotheses and validation approach
36. Active learning strategy
37. Feedback loop design
38. MLOps pipeline diagram
39. Monitoring and retraining plan
40. Expected accuracy improvement roadmap

Day 5: Agentic AI, Skill Concept, Guardrails, Agent-to-Agent Communication and Capstone

Client feedback incorporated: The Skill concept has been added as a dedicated module and connected to agentic AI, MCP, governance, reuse and production readiness.

Theme: Building secure, reliable, observable, multi-agent enterprise AI systems with reusable Skills, governance and production controls.

Module 1: Agentic AI Architecture

- What makes an AI system agentic
- Planner-executor pattern
- Tool-calling agents
- Workflow agents
- Reflection agents
- Supervisor-worker pattern
- Multi-agent collaboration
- Memory
- State management
- Task decomposition
- Human approval gates
- Agent observability
- Failure recovery

Hands-On Lab

Participants design a field-service multi-agent system consisting of a diagnostic agent, manual retrieval agent, maintenance recommendation agent, spare parts agent, report generation agent and human approval agent.

Module 2: Agent-to-Agent Communication

- Why agent-to-agent communication matters
- Agent discovery
- Agent capability description
- Message passing
- Shared context
- Task delegation
- Tool versus agent distinction
- Protocol-based interoperability
- Multi-agent failure modes
- Agent auditability

Hands-On Lab

Participants simulate two agents communicating: a service diagnosis agent and a knowledge retrieval agent. They define agent role, input contract, output contract, allowed tools, failure condition and escalation condition.

Module 3: Skill Concept for Enterprise AI and Agentic Systems

- What an AI/agent Skill is: a reusable capability package for a specific task or workflow
- Difference between prompt, tool, MCP server and Skill
- Skill components: instructions, trigger conditions, examples, schemas, tool bindings, deterministic code snippets, safety rules and test cases
- When to create a Skill instead of a one-off prompt
- Skill registry and reuse across teams
- Skill versioning and approval workflow
- Skill testing and regression checks
- Skill observability: invocation rate, success rate, failure patterns, token cost, and latency
- Security and governance for Skills that call tools or take actions
- Examples: troubleshooting Skill, RAG retrieval Skill, test-case generation Skill, release-note Skill, incident-summary Skill

Hands-On Lab

Teams design one reusable Skill for the client's workflow. They define the skill's purpose, inputs, outputs, examples, allowed tools, guardrails, test cases, and monitoring metrics.

Module 4: Guardrails and Responsible AI Engineering

- Prompt injection
- Data leakage
- Sensitive information disclosure
- Unsafe tool use
- Excessive autonomy
- Model hallucination
- Output validation
- Policy checks
- PII masking
- Role-based response filtering
- Retrieval security
- Human-in-the-loop approval
- Audit trail
- Red teaming
- Safety test cases

Hands-On Lab

Participants attack their own RAG or agent system using adversarial prompts and then implement guardrails.

Module 5: Research-to-Use-Case Translation

- How to read AI research papers quickly
- How to identify usable ideas from research
- How to convert research into engineering experiments
- How to compare research claims with enterprise constraints
- How to run small ablation studies
- How to maintain an AI research backlog
- How to document technical decisions

Hands-On Lab

Participants review one recent paper or framework and map it to a client-relevant use case, including a testable hypothesis.

Module 6: Final Capstone

Each team presents a production-grade redesign of one PoC.

41. Problem statement
42. Baseline accuracy
43. Target success metric
44. Failure taxonomy
45. Hypotheses and validation plan
46. Selected architecture pattern and rationale
47. Target architecture
48. Data architecture
49. RAG or ML/CV pipeline
50. MCP/tool integration design
51. Skill design, if applicable

52. MLOps/LLMOps pipeline
53. Guardrail design
54. Observability plan
55. Feedback loop
56. Monitoring plan
57. 95% improvement roadmap
58. Implementation plan for 30, 60 and 90 days

Daily Improvement Tracking

Each day ends with a scored assignment so that the improvement of every team can be tracked throughout the workshop.

Evaluation Area	Weight
Problem clarity	10%
Metric definition	10%
Hypothesis quality and validation design	10%
Architecture quality and selection rationale	15%
Data strategy	12%
Model/RAG/CV design	13%
MCP, Skill and integration design	8%
Observability and monitoring design	8%
Security and guardrails	7%
Practical implementation quality	7%

Improvement Dashboard

- Baseline score
- Improved score
- Number of failure categories identified
- Number of failure categories reduced
- Hypotheses tested
- Retrieval quality
- Hallucination reduction
- False positive reduction
- False negative reduction
- Human review reduction
- Latency
- Token and infrastructure cost
- MCP/tool-call failures
- Skill invocation success rate
- Security violations found
- Security violations fixed

Suggested Tool Stack for Hands-On Labs

The final tool stack can be adjusted based on the client's internal environment, approved cloud stack and data policy.

Area	Tools / Capabilities
Core Development	Python, FastAPI, Docker, Git, Jupyter/VS Code
Data and Storage	PostgreSQL, object storage, pgvector/Qdrant/Milvus/Pinecone, graph database if Graph RAG is included, DVC or LakeFS
RAG and GenAI	LangChain, LlamaIndex, Semantic Kernel, RAGAS or equivalent RAG evaluation framework, Promptfoo or equivalent prompt regression testing
MCP and Tool Integration	MCP server/client patterns, tool contracts, resource exposure, secure connector design, tool-call audit logging
Observability	OpenTelemetry-style traces, LangSmith/Langfuse/Phoenix/Arize-style observability concepts, retrieval traces, token/cost/latency dashboards

Area	Tools / Capabilities
MLOps	MLflow, Airflow/Dagster/Prefect, Evidently AI or equivalent monitoring tool, model registry, experiment tracking
Computer Vision	OpenCV, PyTorch, YOLO/Detectron/ViT-based models depending on use case, Label Studio or CVAT
Agentic AI	LangGraph, Semantic Kernel, AutoGen-style multi-agent design, MCP-style tool integration, A2A-style agent communication concepts
Skills	Skill templates, Skill registry concept, reusable instructions/examples/code/tool bindings, Skill test cases and governance workflow
Security and Guardrails	Input validation, output validation, PII masking, role-based response filtering, prompt injection test cases, secure tool execution, audit logging

Optional Advanced Add-On: 2-Week Post-Training Mentoring

To convert training into measurable business impact, a 2-week post-training mentoring track is recommended.

Week 1

- Review existing Client's PoCs
- Select two high-priority PoCs
- Redesign architecture
- Build evaluation dataset
- Define target metrics
- Create hypotheses and an experiment plan
- Identify data gaps

Week 2

- Implement improved RAG/CV/MLOps pipeline
- Set up evaluation dashboard
- Set up observability dashboard
- Build a feedback loop
- Add guardrails
- Create Skill prototype where applicable
- Prepare production readiness checklist
- Present 90-day implementation roadmap

Final Client-Facing Program Promise

After this program, participants will not merely understand AI concepts. They will be able to diagnose, redesign, evaluate, secure, observe and scale enterprise AI systems. They will learn how to move beyond isolated PoCs and build production-grade AI applications with measurable accuracy, reliability, governance and business impact.

The training will create a practical engineering pathway for moving current AI PoCs from around 60% quality toward 95%+ production-grade performance, subject to data quality, business metric definition, availability of ground truth, architecture maturity, implementation discipline and continuous feedback loops.